



Awk

This is a one page quick reference cheat sheet to the GNU awk, which covers commonly used awk expressions and commands.

Getting Started

[Have a try](#)

```
$ awk -F: '{print $1, $NF}' /etc/passwd
```

<code>-F:</code>	Colon as a separator
<code>{...}</code>	Awk program
<code>print</code>	Prints the current record
<code>\$1</code>	First field
<code>\$NF</code>	Last field
<code>/etc/passwd</code>	Input data file

[Awk program](#)

```
BEGIN      {<initializations>}
  <pattern 1> {<program actions>}
  <pattern 2> {<program actions>}
  ...
END        {< final actions >}
```

Example

```
awk ' '
```

```
BEGIN { print "\n>>>Start" }
!/(login|shutdown)/ { print NR, $0 }
END { print "<<<END\n" }
' /etc/passwd
```

Variables

	\$1	\$2/\${NF-1}	\$3/\$NF
	▼	▼	▼
\$0/NR ►	ID	WEBSITE	URI
\$0/NR ►	1	quickref.me	awk
\$0/NR ►	2	google.com	25

```
# First and last field
awk -F: '{print $1,$NF}' /etc/passwd

# With line number
awk -F: '{print NR, $0}' /etc/passwd

# Second last field
awk -F: '{print ${NF-1}}' /etc/passwd

# Custom string
awk -F: '{print $1 "=" $6}' /etc/passwd
```

See: [Variables](#)

Awk program examples

```
awk 'BEGIN {print "hello world"}'           # Prints "hello world"
awk -F: '{print $1}' /etc/passwd           # -F: Specify field separator

# /pattern/ Execute actions only for matched pattern
awk -F: '/root/ {print $1}' /etc/passwd

# BEGIN block is executed once at the start
awk -F: 'BEGIN { print "uid"} { print $1 }' /etc/passwd

# END block is executed once at the end
awk -F: '{print $1} END { print "-done-"}' /etc/passwd
```

Conditions

```
awk -F: '$3>30 {print $1}' /etc/passwd
```

See: [Conditions](#)

Generate 1000 spaces

```
awk 'BEGIN{
    while (a++ < 1000)
        s=s " ";
    print s
}'
```

See: [Loops](#)

Arrays

```
awk 'BEGIN {
    fruits["mango"] = "yellow";
    fruits["orange"] = "orange"
    for(fruit in fruits) {
        print "The color of " fruit " is " fruits[fruit]
    }
}'
```

See: [Arrays](#)

Functions

```
# => 5
awk 'BEGIN{print length("hello")}'
# => HELLO
awk 'BEGIN{print toupper("hello")}'
# => hel
awk 'BEGIN{print substr("hello", 1, 3)}'
```

See: [Functions](#)

Awk Variables

Build-in variables

<code>\$0</code>	Whole line
<code>\$1, \$2...\$NF</code>	First, second... last field
<code>NR</code>	Number of Records
<code>NF</code>	Number of Fields
<code>OFS</code>	Output Field Separator (default " ")
<code>FS</code>	input Field Separator (default " ")
<code>ORS</code>	Output Record Separator (default "\n")
<code>RS</code>	input Record Separator (default "\n")
<code>FILENAME</code>	Name of the file

Expressions

<code>\$1 == "root"</code>	First field equals root
<code>{print \$(NF-1)}</code>	Second last field
<code>NR!=1{print \$0}</code>	From 2th record
<code>NR > 3</code>	From 4th record
<code>NR == 1</code>	First record
<code>END{print NR}</code>	Total records
<code>BEGIN{print OFMT}</code>	Output format
<code>{print NR, \$0}</code>	Line number
<code>{print NR " " \$0}</code>	Line number (tab)
<code>{\$1 = NR; print}</code>	Replace 1th field with line number
<code>\$NF > 4</code>	Last field > 4

```
NR % 2 == 0
```

Even records

```
NR==10, NR==20
```

Records 10 to 20

```
BEGIN{print ARGC}
```

Total arguments

```
ORS=NR%5?" ":"\n"
```

Concatenate records

Examples

Print sum and average

```
awk -F: '{sum += $3}
        END { print sum, sum/NR }
' /etc/passwd
```

Printing parameters

```
awk 'BEGIN {
    for (i = 1; i < ARGC; i++)
        print ARGV[i] }' a b c
```

Output field separator as a comma

```
awk 'BEGIN { FS=":";OFS=","}
      {print $1,$2,$3,$4}' /etc/passwd
```

Position of match

```
awk 'BEGIN {
    if (match("One Two Three", "Tw"))
        print RSTART }'
```

Length of match

```
awk 'BEGIN {
    if (match("One Two Three", "re"))
        print RLENGTH }'
```

Environment Variables

ARGC

Number or arguments

ARGV	Array of arguments
FNR	File Number of Records
OFMT	Format for numbers (default "%.6g")
RSTART	Location in the string
RLENGTH	Length of match
SUBSEP	Multi-dimensional array separator (default "\034")
ARGIND	Argument Index

GNU awk only

ENVIRON	Environment variables
IGNORECASE	Ignore case
CONVFMT	Conversion format
ERRNO	System errors
FIELDWIDTHS	Fixed width fields

Defining variable

```
awk -v var1="Hello" -v var2="World" '
    END {print var1, var2}
' </dev/null
```

Use shell variables

```
awk -v varName="$PWD" '
    END {print varName}' </dev/null
```

Awk Operators

Operators

<code>{print \$1}</code>	First field
<code>\$2 == "foo"</code>	Equals
<code>\$2 != "foo"</code>	Not equals
<code>"foo" in array</code>	In array
Regular expression	
<code>/regex/</code>	Line matches
<code>!/regex/</code>	Line not matches
<code>\$1 ~ /regex/</code>	Field matches
<code>\$1 !~ /regex/</code>	Field not matches
More conditions	
<code>(\$2 <= 4 \$3 < 20)</code>	Or
<code>(\$1 == 4 && \$3 < 20)</code>	And

Operations

Arithmetic operations		
<code>+</code>	<code>-</code>	<code>*</code>
<code>/</code>	<code>%</code>	<code>++</code>
<code>--</code>		
Shorthand assignments		
<code>+=</code>	<code>-=</code>	<code>*=</code>
<code>/=</code>	<code>%=</code>	
Comparison operators		
<code>==</code>	<code>!=</code>	<code><</code>
<code>></code>	<code><=</code>	<code>>=</code>

Examples

```
awk 'BEGIN {
```

```
if ("foo" ~ "^fo+$")
    print "Foocy!";
}'
```

Not match

```
awk 'BEGIN {
    if ("boo" !~ "^fo+$")
        print "Boo!";
}'
```

if in array

```
awk 'BEGIN {
    assoc["foo"] = "bar";
    assoc["bar"] = "baz";
    if ("foo" in assoc)
        print "Foocy!";
}'
```

Awk Functions

Common functions

<code>index(s,t)</code>	Position in string s where string t occurs, 0 if not found
<code>length(s)</code>	Length of string s (or \$0 if no arg)
<code>rand</code>	Random number between 0 and 1
<code>substr(s,index,len)</code>	Return len-char substring of s that begins at index (counted from 1)
<code>srand</code>	Set seed for rand and return previous seed
<code>int(x)</code>	Truncate x to integer value
<code>split(s,a,fs)</code>	Split string s into array a split by fs, returning length of a
<code>match(s,r)</code>	Position in string s where regex r occurs, or 0 if not found
<code>sub(r,t,s)</code>	Substitute t for first occurrence of regex r in string s (or \$0 if s not given)
<code>gsub(r,t,s)</code>	Substitute t for all occurrences of regex r in string s

`system(cmd)`

Execute cmd and return exit status

`tolower(s)`

String s to lowercase

`toupper(s)`

String s to uppercase

`getline`

Set \$0 to next input record from current input file.

User defined function

```
awk '
# Returns minimum number
function find_min(num1, num2){
    if (num1 < num2)
        return num1
    return num2
}
# Returns maximum number
function find_max(num1, num2){
    if (num1 > num2)
        return num1
    return num2
}
# Main function
function main(num1, num2){
    result = find_min(num1, num2)
    print "Minimum =", result

    result = find_max(num1, num2)
    print "Maximum =", result
}
# Script execution starts here
BEGIN {
    main(10, 60)
}
'
```

Awk Arrays

Array with index

```
awk 'BEGIN {
    arr[0] = "foo";
    arr[1] = "bar";
    print(arr[0]); # => foo
    delete arr[0];
    print(arr[0]); # => ""
}'
```

Array with key

```
awk 'BEGIN {
    assoc["foo"] = "bar";
    assoc["bar"] = "baz";
    print("baz" in assoc); # => 0
    print("foo" in assoc); # => 1
}'
```

Array with split

```
awk 'BEGIN {
    split("foo:bar:baz", arr, ":");
    for (key in arr)
        print arr[key];
}'
```

Array with asort

```
awk 'BEGIN {
    arr[0] = 3
    arr[1] = 2
    arr[2] = 4
    n = asort(arr)
    for (i = 1; i <= n ; i++)
        print(arr[i])
}'
```

Multi-dimensional

```
awk 'BEGIN {
    multidim[0,0] = "foo";
    multidim[0,1] = "bar";
    multidim[1,0] = "baz";
    multidim[1,1] = "boo";
}'
```

```
awk 'BEGIN {
    array[1,2]=3;
    array[2,3]=5;
    for (comb in array) {
        split(comb,sep,SUBSEP);
        print sep[1], sep[2],
        array[sep[1],sep[2]]
    }
}'
```

Awk Conditions

```
awk -v count=2 'BEGIN {
    if (count == 1)
        print "Yes";
    else
        print "Huh?";
}'
```

Ternary operator

```
awk -v count=2 'BEGIN {
    print (count==1) ? "Yes" : "Huh?";
}'
```

```
awk 'BEGIN {
    assoc["foo"] = "bar";
    assoc["bar"] = "baz";
    if ("foo" in assoc)
        print "Fooley!";
}'
```

Not exists

```
awk 'BEGIN {
```

```

assoc["foo"] = "bar";
assoc["bar"] = "baz";
if ("Huh" in assoc == 0 )
    print "Huh!";
}'

```

switch

```

awk -F: '{
    switch (NR * 2 + 1) {
        case 3:
        case "11":
            print NR - 1
            break

        case /2[[:digit:]]+/:
            print NR

        default:
            print NR + 1

        case -1:
            print NR * -1
    }
}' /etc/passwd

```

Awk Loops

for...i

```

awk 'BEGIN {
    for (i = 0; i < 10; i++)
        print "i=" i;
}'

```

Powers of two between 1 and 100

```

awk 'BEGIN {
    for (i = 1; i <= 100; i *= 2)
        print i
}'

```

```
awk 'BEGIN {
    assoc["key1"] = "val1"
    assoc["key2"] = "val2"
    for (key in assoc)
        print assoc[key];
}'
```

Arguments

```
awk 'BEGIN {
    for (argnum in ARGV)
        print ARGV[argnum];
}' a b c
```

Reverse records

```
awk -F: '{ x[NR] = $0 }
END {
    for (i = NR; i > 0; i--)
        print x[i]
}
' /etc/passwd
```

Reverse fields

```
awk -F: '{
    for (i = NF; i > 0; i--)
        printf("%s ", $i);
    print ""
}' /etc/passwd
```

Sum by record

```
awk -F: '{
    s=0;
    for (i = 1; i <= NF; i++)
        s += $i;
    print s
}' /etc/passwd
```

Sum whole file

```
awk -F: '
    {for (i = 1; i <= NF; i++)
        s += $i;
    };
    END{print s}
}' /etc/passwd
```

while

```
awk 'BEGIN {
    while (a < 10) {
        print "- " " concatenation: " a
        a++;
    }
}'
```

do...while

```
awk '{
    i = 1
    do {
        print $0
        i++
    } while (i <= 5)
}' /etc/passwd
```

Break

```
awk 'BEGIN {
    break_num = 5
    for (i = 0; i < 10; i++) {
        print i
        if (i == break_num)
            break
    }
}'
```

Continue

```
awk 'BEGIN {
    for (x = 0; x <= 10; x++) {
        if (x == 5 || x == 6)
            continue
        printf "%d ", x
    }
}
```

```
print ""
}'
```

Awk Formatted Printing

Usage

Right align

```
awk 'BEGIN{printf "|%10s|\n", "hello"}'

|      hello|
```

Left align

```
awk 'BEGIN{printf "|%-10s|\n", "hello"}'

|hello      |
```

Common specifiers

c	ASCII character
d	Decimal integer
e, E, f	Floating-point format
o	Unsigned octal value
s	String
%	Literal %

Space

```
awk -F: '{
    printf "%-10s %s\n", $1, $(NF-1)
}' /etc/passwd | head -n 3
```

Outputs

```
root      /root
```

```
bin      /bin
daemon   /sbin
```

Header

```
awk -F: 'BEGIN {
    printf "%-10s %s\n", "User", "Home"
    printf "%-10s %s\n", "----", "----"}
    { printf "%-10s %s\n", $1, $(NF-1) }
' /etc/passwd | head -n 5
```

Outputs

```
User      Home
----      ----
root       /root
bin        /bin
daemon     /sbin
```

Miscellaneous

Regex Metacharacters

\	^	\$
.	[	]
	(	)
*	+	?

Escape Sequences

\b	Backspace
\f	Form feed
\n	Newline (line feed)
\r	Carriage return
\t	Horizontal tab

Run script

```
$ cat demo.awk
#!/usr/bin/awk -f
BEGIN { x = 23 }
      { x += 2 }
END   { print x }
$ awk -f demo.awk /etc/passwd
69
```

Also see

[The GNU Awk User's Guide](http://www-zeuthen.desy.de) (www-zeuthen.desy.de)

[AWK cheatsheet](https://gist.github.com) (gist.github.com)

Related Cheatsheet

Emacs Cheatsheet

Quick Reference

Grep Cheatsheet

Quick Reference

Markdown Cheatsheet

Quick Reference

Sed Cheatsheet

Quick Reference

Recent Cheatsheet

Remote Work Revolution Cheatsheet

Quick Reference

Homebrew Cheatsheet

Quick Reference

PyTorch Cheatsheet

Quick Reference

Taskset Cheatsheet

Quick Reference

